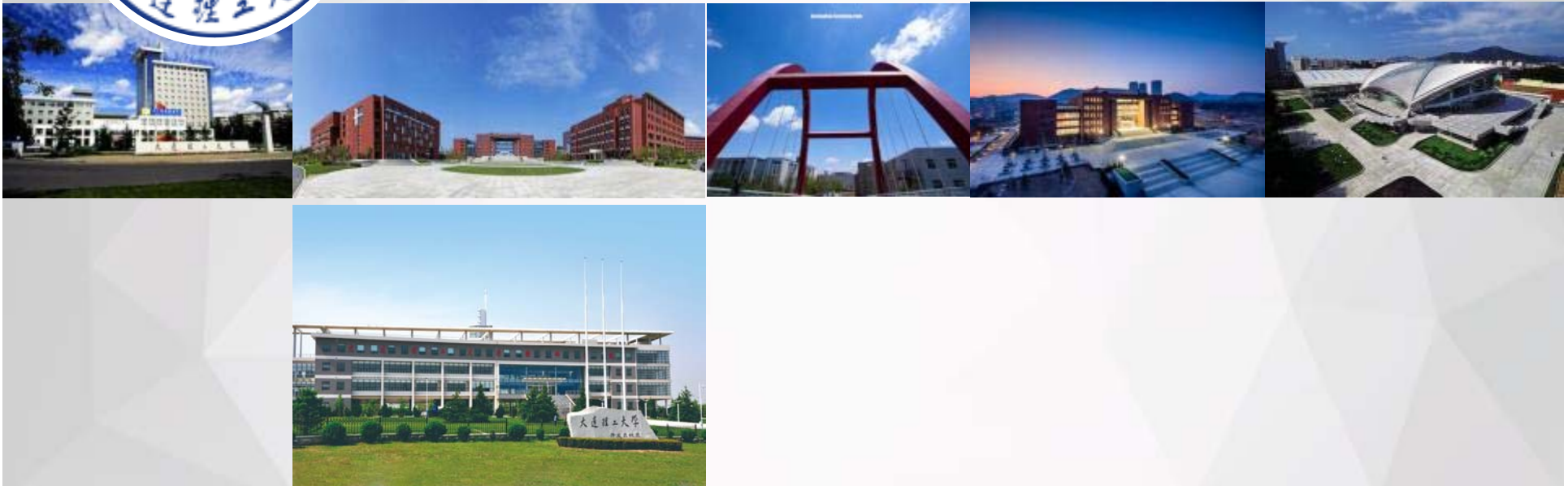




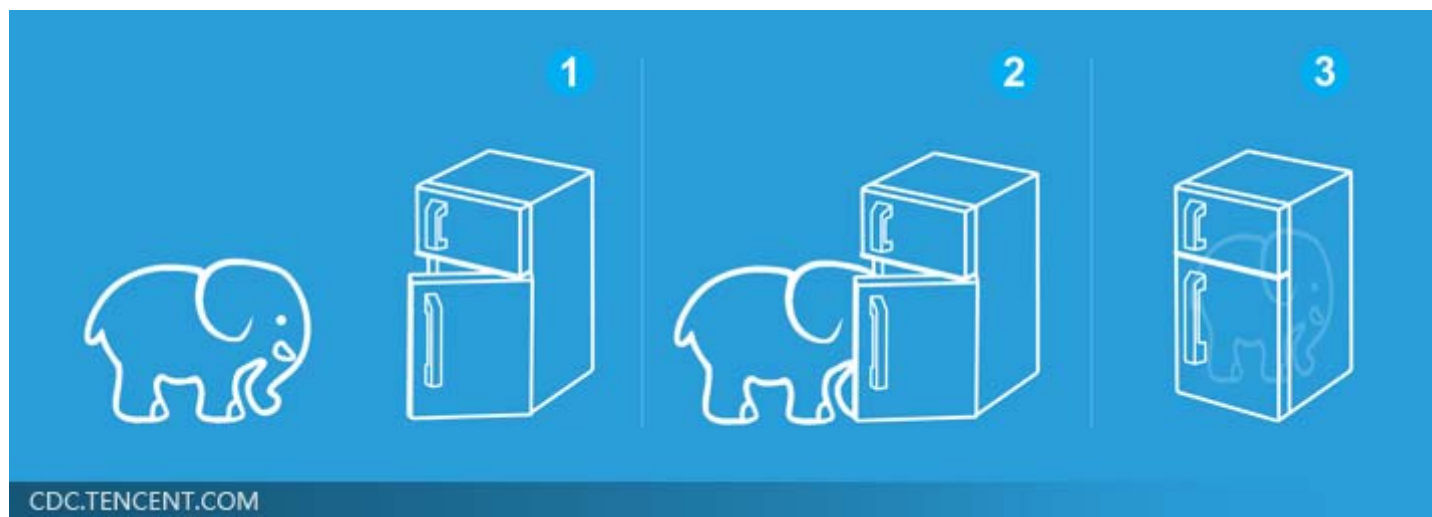
深度神经网络及训练I

DNNs and Training





Deep Learning is so simple



- 01 网络结构 (Network Structure)
- 02 损失函数 (Loss Function)
- 03 模型优化 (Model Optimazation)
- 04 超参技巧 (Hyper-parameter Tricks)
- 05 模型微调 (Model Fine-tuning)
- 06 小结(Summary)





01

PART ONE

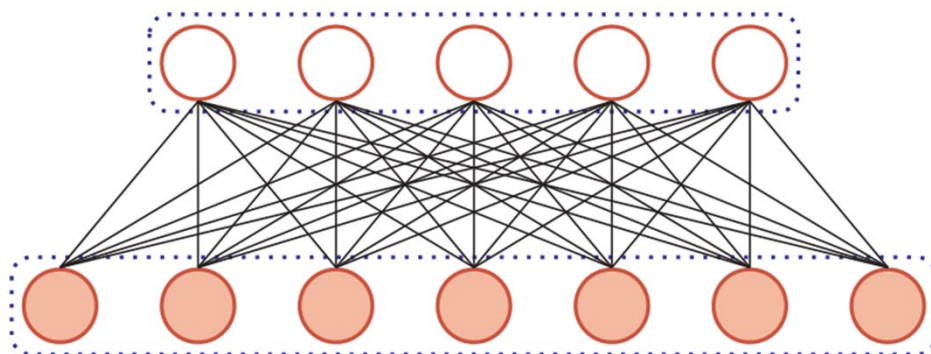
网络结构

Network Structure



■ 全连接网络(Fully Connected Network)

结构表示如下：



➤ 权重矩阵的参数量巨大

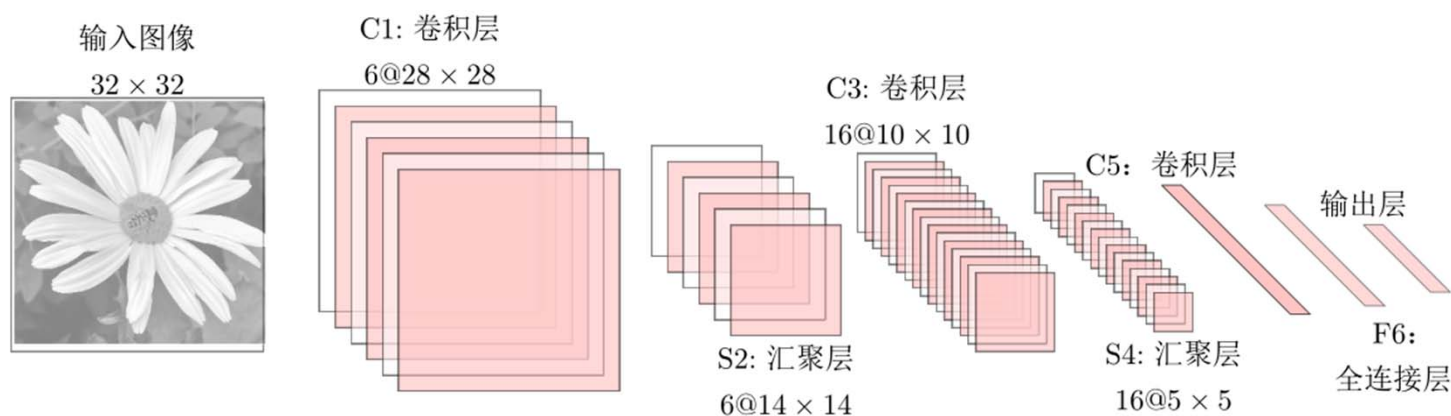
➤ 局部不变性特征

自然图像中的物体都具有局部不变性特征，如尺度缩放、平移、旋转等

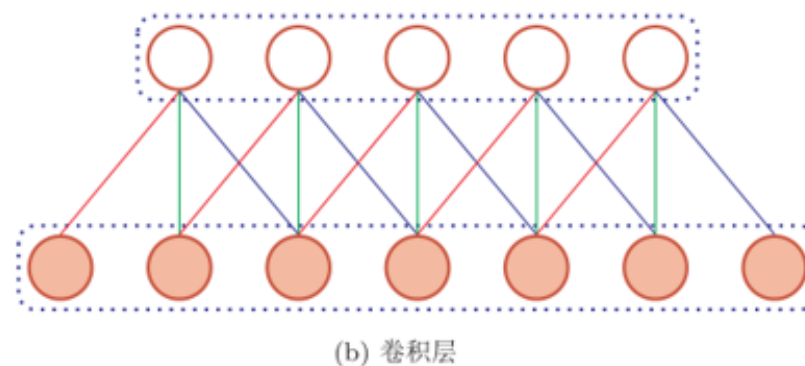
➤ 全连接网络很难提取这些局部不变特征

卷积神经网络(Convolutional Neural Network)

结构表示如下:

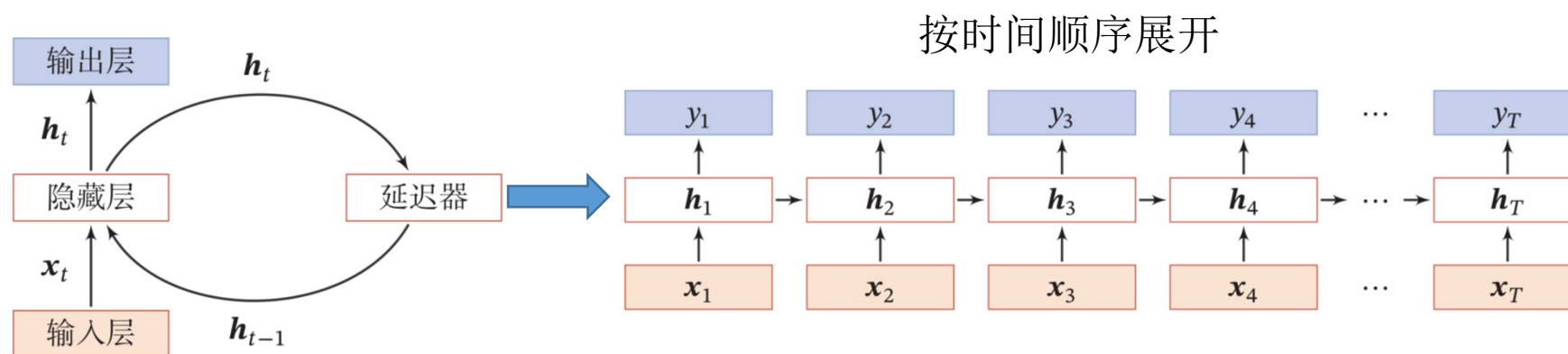


- 受生物学上感受野(Receptive Field)的机制而提出
- 局部连接
- 权重共享
- 平移不变(空间下采样)



■ 循环神经网络(Recurrent Neural Network)

结构表示如下:



- 使用带自反馈的神经元，能够处理任意长度的时序数据
- 比前馈神经网络更加符合生物神经网络的结构
- 网络具有自适应记忆能力



PART TWO

损失函数

Loss Function



■ 损失函数(Loss Function)

- 0-1损失函数(Zero-one Loss)
- 绝对值损失函数(Absolute Loss)
- 对数损失函数(Log loss)
- 平方损失函数(Square Loss)
- 指数损失函数(Exponential Loss)
- 合页损失函数(Hinge Loss)
- 感知损失函数(Perceptron Loss)
- 交叉熵损失函数 (Cross-entropy Loss)
- 聚焦损失函数(Focal Loss)

■ 0-1损失函数(Zero-one Loss)

➤ 0-1损失函数是指预测值和目标值不相等为1， 否则为0:

$$L(Y, f(X)) = \begin{cases} 1, Y \neq f(X) \\ 0, Y = f(X) \end{cases}$$

➤ 特点:

(1) 0-1损失函数直接对应分类判断错误的个数，但是它是一个非凸函数，不太适用

(2) 相等条件太过严格，因此可以放宽条件，即满足

$$L(Y, f(X)) = \begin{cases} 1, |Y - f(X)| \geq T \\ 0, |Y - f(X)| < T \end{cases}$$

■ 绝对值损失函数(Absolute Loss)

➤ 绝对值损失函数是计算预测值与目标值的差的绝对值：

$$L(Y, f(x)) = |Y - f(x)|$$

➤ 特点：

- (1) 可用作回归任务，但是它是一个非凸函数，不太适用
- (2) 损失函数在训练中不稳定，对于底层视觉任务比较有效



■ 对数损失函数(Log loss)

➤ 对数损失函数是对预测得分取对数，其形式如下：

$$L(Y, P(Y|X)) = -\log P(Y|X)$$

➤ 特点：

- (1) 对数损失函数能非常好的表征概率分布，在很多场景尤其是多分类，如果需要知道结果属于每个类别的置信度，那它非常适合
- (2) 鲁棒性不强，对噪声更敏感
- (3) 逻辑回归的损失函数就是对数损失函数

■ 平方损失函数(Square Loss)

➤ 平方损失函数的形式如下：

$$L(Y|f(X)) = \sum_N (Y - f(X))^2$$

➤ 特点：

- (1) 经常应用与回归问题
- (2) 鲁棒性不强，对噪声敏感
- (3) 最小二乘有闭形式解

■ 指数损失函数(Exponential Loss)

- 指数损失函数的标准形式如下：

$$L(Y|f(X)) = \exp[-yf(x)]$$

- 特点：

(1)对离群点、噪声非常敏感；经常用在AdaBoost算法

■合页损失函数(Hinge Loss)

➤ 合页损失函数标准形式如下：

$$L(y, f(x)) = \max(0, 1 - yf(x))$$

➤ 特点：

(1) 合页损失函数表示如果被分类正确，损失为0，否则损失就为 $1 - yf(x)$
SVM就是使用这个损失函数

(2) 一般 $f(x)$ 的预测值，在-1到1之间， y 是目标值(-1或1)。其含义是， $f(x)$ 的值在-1和+1之间就可以了，并不鼓励 $|f(x)| > 1$ ，即并不鼓励分类器过度自信，让某个正确分类的样本距离分割线超过1并不会有任何奖励，从而使分类器可以更专注于整体的误差

(3) 鲁棒性相对较高，对异常点、噪声不敏感，但它没太好的概率解释

■感知损失函数(Perceptron Loss)

➤ 感知损失函数的标准形式如下：

$$L(y, f(x)) = \max(0, -f(x))$$

➤ 特点：

(1) 它是合页损失函数的一个变种，合页损失函数对判定边界附近的点惩罚力度很高。而感知损失函数只要判定样本的正确类别，不管其判定边界的距离

(2) 它比合页损失函数简单，但不考量样本边界，所以模型的泛化能力没合页损失函数强

■交叉熵损失函数 (Cross-entropy Loss)

➤ 交叉熵损失函数的标准形式如下：

$$C = -\frac{1}{n} \sum_x [y \ln a + (1 - y) \ln(1 - a)]$$

➤ 特点：

(1) 本质上也是一种对数似然函数，可用于二分类和多分类任务中
二分类问题中的损失函数(输入数据是softmax或sigmoid函数的输出)：

$$loss = -\frac{1}{n} \sum_x [y \ln a + (1 - y) \ln(1 - a)]$$

多分类问题中的损失函数(输入数据是softmax或sigmoid函数的输出)：

$$loss = -\frac{1}{n} \sum_i y_i \ln a_i$$

■交叉熵损失函数 (Cross-entropy Loss)

➤ 交叉熵损失函数的标准形式如下：

$$C = -\frac{1}{n} \sum_x [y \ln a + (1 - y) \ln(1 - a)]$$

➤ 特点：

(2) 当使用softmax 或 sigmoid作为激活函数的时候，常用交叉熵损失函数而不用均方误差损失函数，因为它可以完美解决平方损失函数权重更新过慢的问题，具有“误差大的时候，权重更新快；误差小的时候，权重更新慢”的良好性质

■ 聚焦损失函数(Focal Loss)

- 聚焦损失函数是在交叉熵损失函数基础上进行的修改

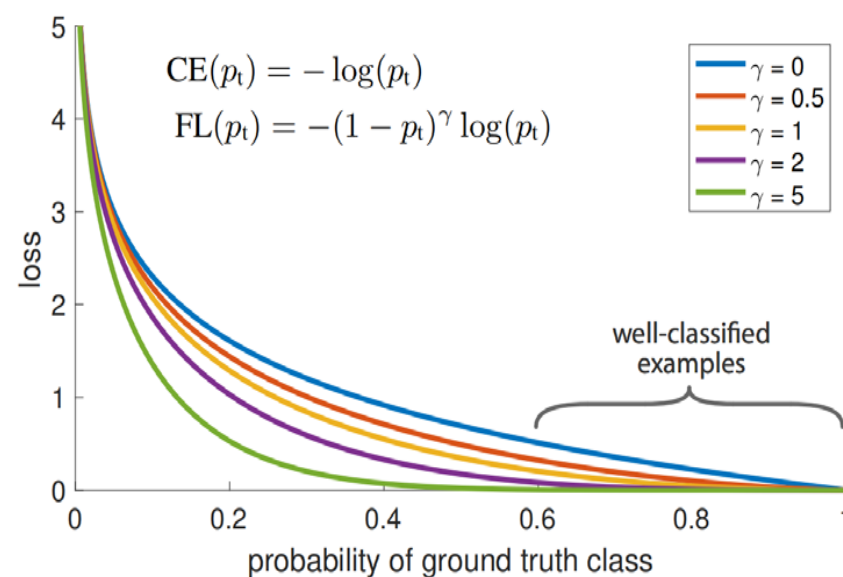
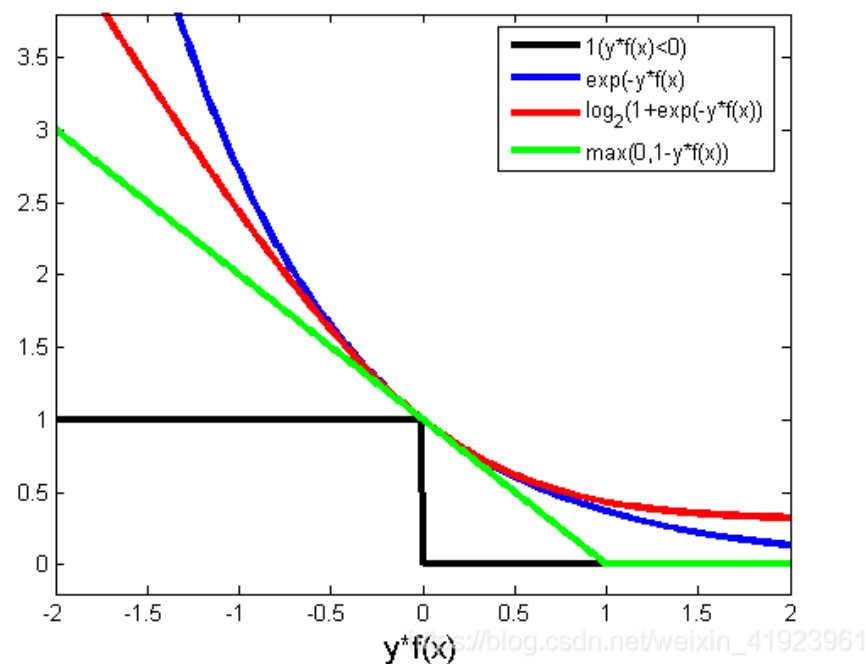
$$\text{FL}(p_t) = -(1 - p_t)^\gamma \log(p_t)$$

- 特点:

- (1) 解决目标检测问题中训练集正负样本极度不平衡问题
- (2) 自适应样本加权: 容易分类样本权重降低, 促进分类器更关注困难样本



损失函数曲线(Loss Function Curve)





PART THREE

模型优化

Model Optimazation



■ 学习准则 (Learning Rule)

▶ 期望风险未知，通过经验风险近似

▶ 训练数据： $\mathcal{D} = \{x^{(n)}, y^{(n)}\}, i \in [1, N]$

$$\mathcal{R}_{\mathcal{D}}^{emp}(\theta) = \frac{1}{N} \sum_{n=1}^N \mathcal{L}(y^{(n)}, f(x^{(n)}, \theta))$$

▶ 经验风险最小化

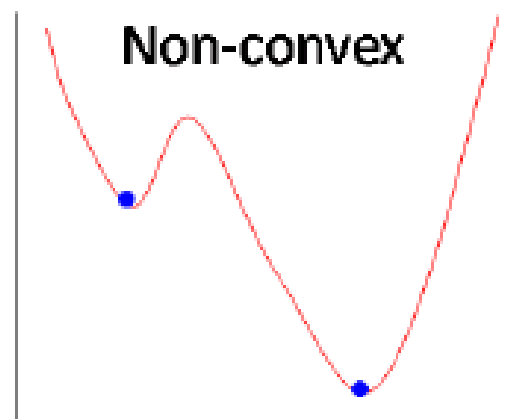
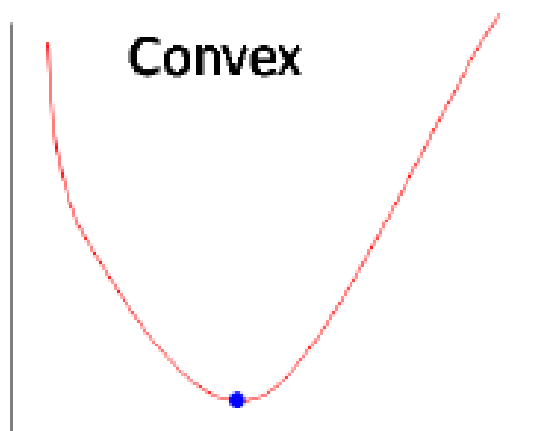
▶ 在选择合适的风险函数后，我们寻找一个参数 θ^* ，使得经验风险函数最小化。

$$\theta^* = \arg \min_{\theta} \mathcal{R}_{\mathcal{D}}^{emp}(\theta)$$

▶ 机器学习问题转化成为一个最优化问题

■ 学习准则 (Learning Rule)

➤ 神经网络的学习转化为一个高阶非凸优化问题



$$\min_{\mathbf{x}} f(\mathbf{x})$$

■ 优化问题 (Optimization Problem)

➤ 难点

参数过多，影响训练

非凸优化问题：存在局部最优而非全局最优解，影响迭代
梯度消失问题，下层参数难调

参数解释起来困难

➤ 需求

计算资源要大

数据要多

算法效率要好：即收敛快



■ 优化问题 (Optimization Problem)

➤ 非凸问题 (Non-convex Problem)

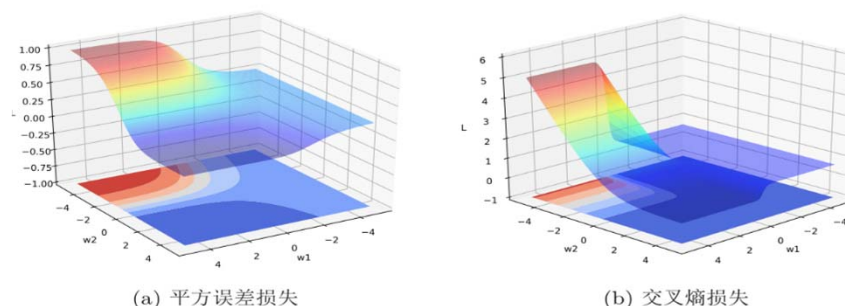
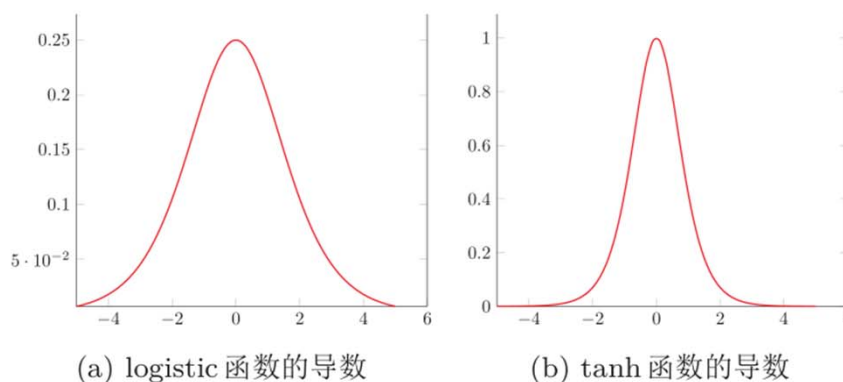


图 4.9 神经网络 $y = \sigma(w_2 \sigma(w_1 x))$ 的损失函数

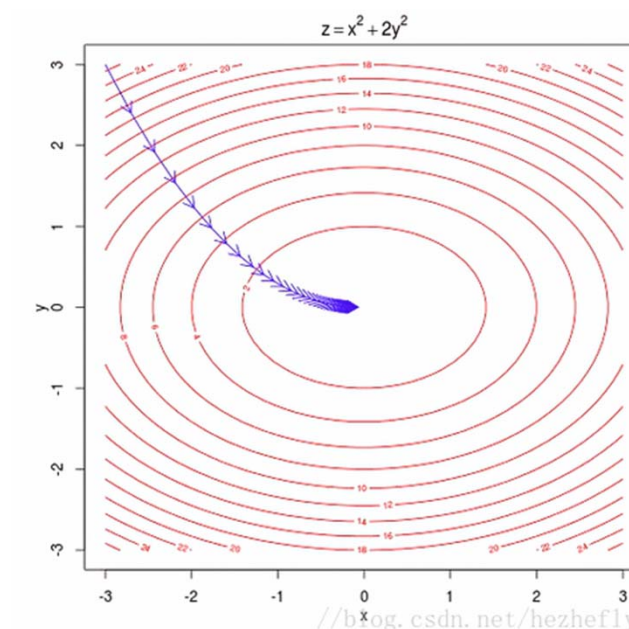
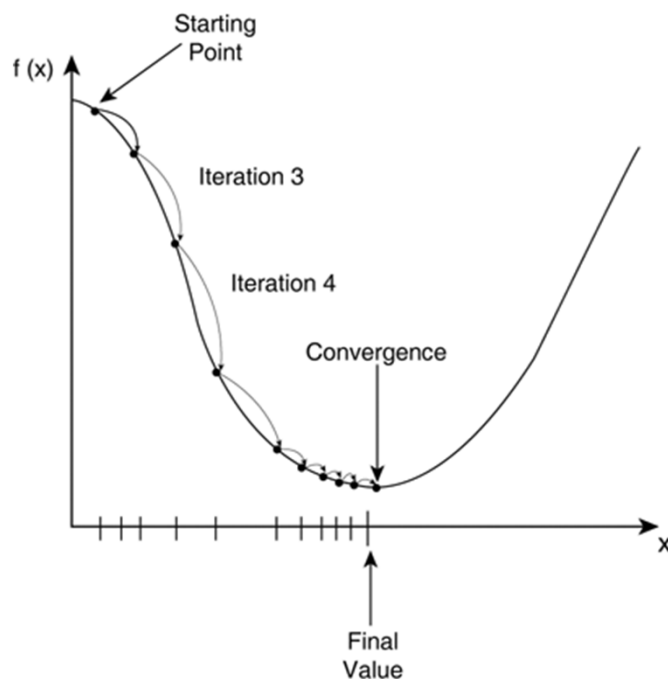
➤ 梯度消失问题 (Vanishing Gradient Problem)

$$y = f^5(f^4(f^3(f^2(f^1(x)))))$$

$$\frac{\partial y}{\partial x} = \frac{\partial f^1}{\partial x} \frac{\partial f^2}{\partial f^1} \frac{\partial f^3}{\partial f^2} \frac{\partial f^4}{\partial f^3} \frac{\partial f^5}{\partial f^4}$$



■ 梯度下降法(Gradient Descent)



$$\begin{aligned}\theta_{t+1} &= \theta_t - \alpha \frac{\partial \mathcal{R}(\theta)}{\partial \theta_t} \\ &= \theta_t - \alpha \frac{1}{N} \sum_{i=1}^N \frac{\partial \mathcal{L}(\theta_t; x^{(i)}, y^{(i)})}{\partial \theta}.\end{aligned}$$

搜索步长 α 中也叫作学习率(Learning Rate)





04

PART Four

超参技巧

Hyper-parameter Tricks



■ 超参数(Hyper-parameter)

➤ 超参数

层数 L

每层神经元个数 N

激活函数 F

学习率 η

正则化系数 λ

批次大小 B

迭代次数 M

动量参数 ε

➤ 选择方法

网格搜索

随机搜索

贝叶斯优化

动态资源分配

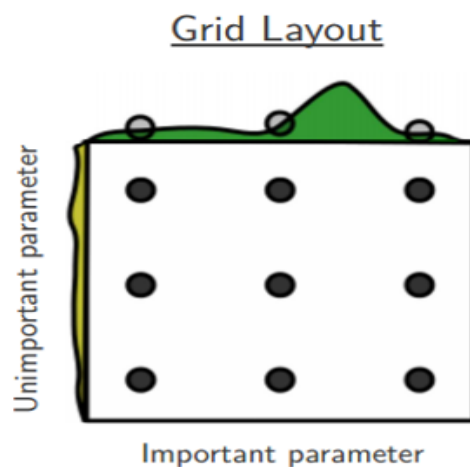
神经架构搜索

■ 网格搜索(Grid Search)

- 假设总共有 K 个超参数，第 k 个超参数可以取 m_k 个值
- 如果参数的取值是连续的，可将参数离散化，以一定的间隔比例，选择几个经验值，比如学习率 η ，我们可以设置

$$\eta \in \{0.01, 0.02, 0.04, 0.06\}$$

超参数可以有 $m_1 \times m_2 \times \dots \times m_K$ 个取值组合

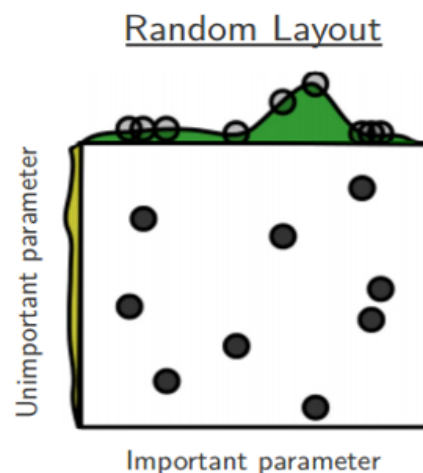


■ 随机搜索(Random Search)

- 假设总共有 K 个超参数，第 k 个超参数可以取 m_k 个值
- 如果参数的取值是连续的，可将参数离散化，**随机**选择几个经验值，比如学习率 η ，我们可以设置

$$\eta \in \{0.01, 0.1, 0.5, 1.0\}$$

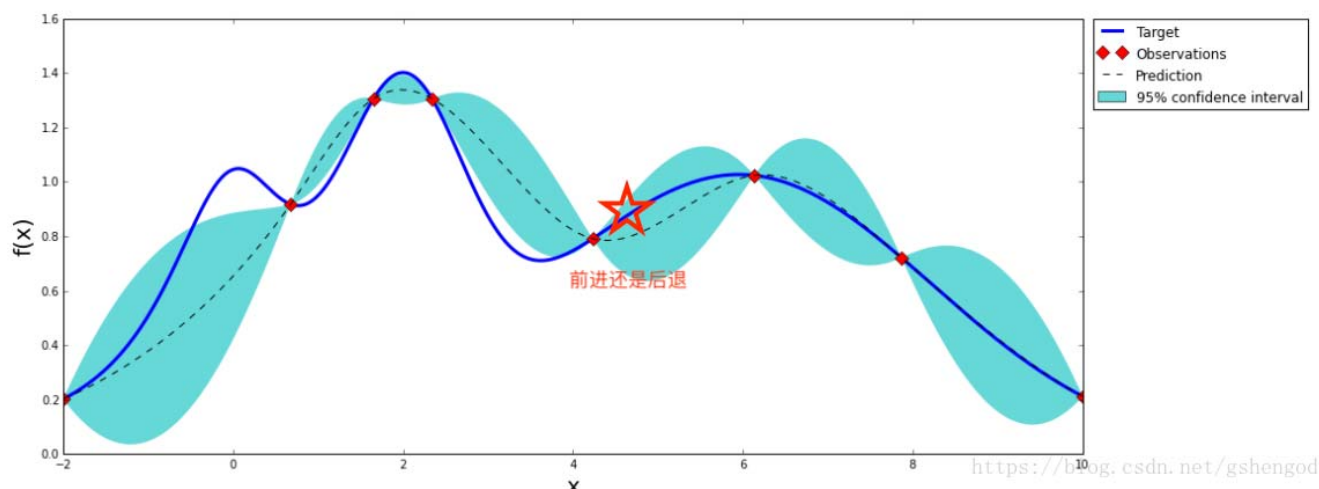
超参数可以有 $m_1 \times m_2 \times \dots \times m_K$ 个取值组合





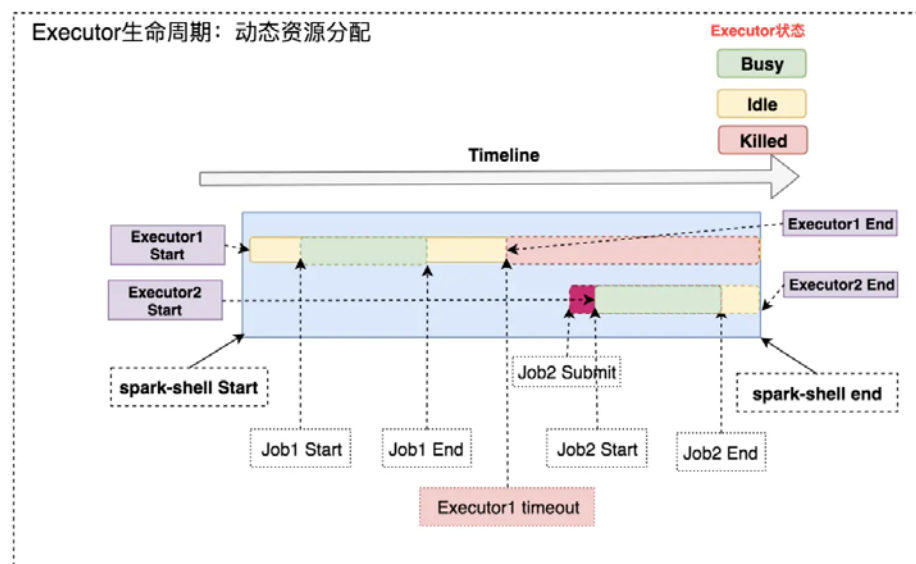
■ 贝叶斯优化(Bayesian Optimization)

- 贝叶斯优化的重要基础理论就是不断通过先验点去预测后验知识
- 把横轴看作是参数组合 X ，纵轴看作是这个参数的结果 Y 可以通过已经构建的曲线，找到曲线上升的方向，从而在这个方向上继续探索，这样就可以大概率拿到更好的结果



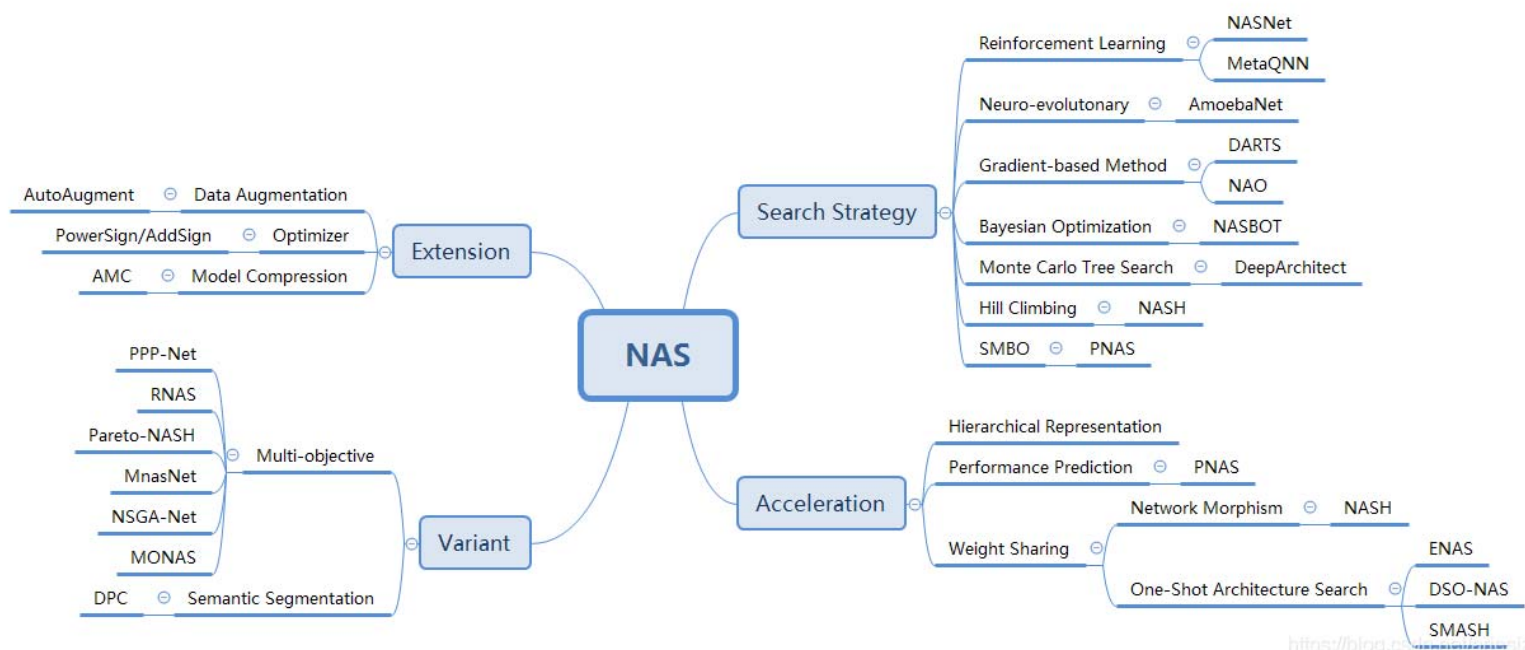
■ 动态资源分配(Dynamic Resource Allocation)

- 可以通过一组超参数的学习曲线来预估这组超参数是否有希望得到比较好的结果
- 如果一组超参数配置的学习曲线不收敛或者收敛比较差，可以应用停止策略终止当前训练
- 动态资源分配的一种有效方法是逐次减半



神经架构搜索(Neural Architecture Search)

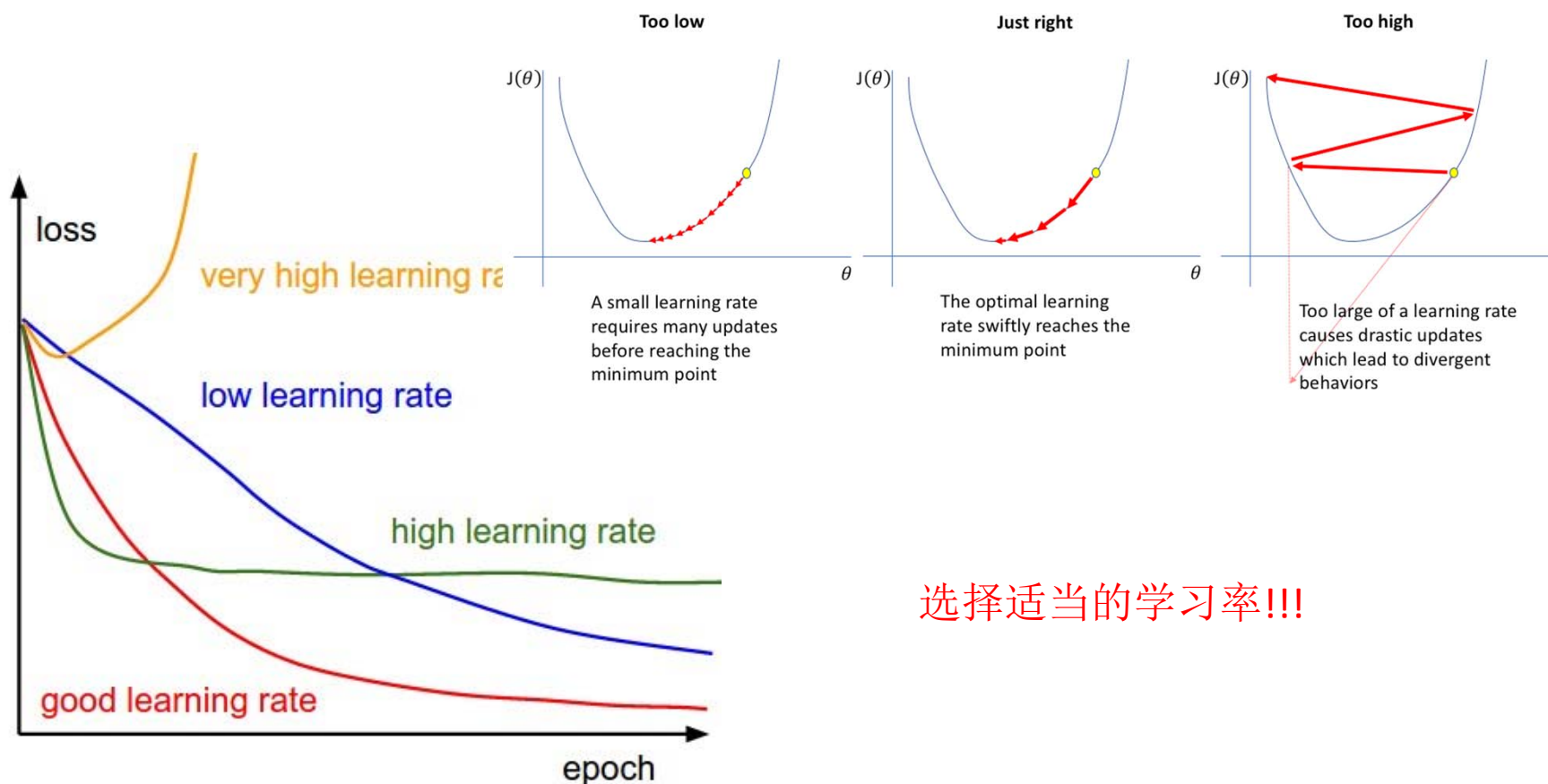
- 以上超参数优化方法都是在固定(或变化比较小)的超参数空间中进行最优配置搜索
- NAS通过神经网络来自动实现网络架构的设计;利用元学习,可以自适应地搜索约定条件的整个参数空间



https://blog.csdn.net/qq_37607579/article/details/84444444

学习率(Learning Rate)

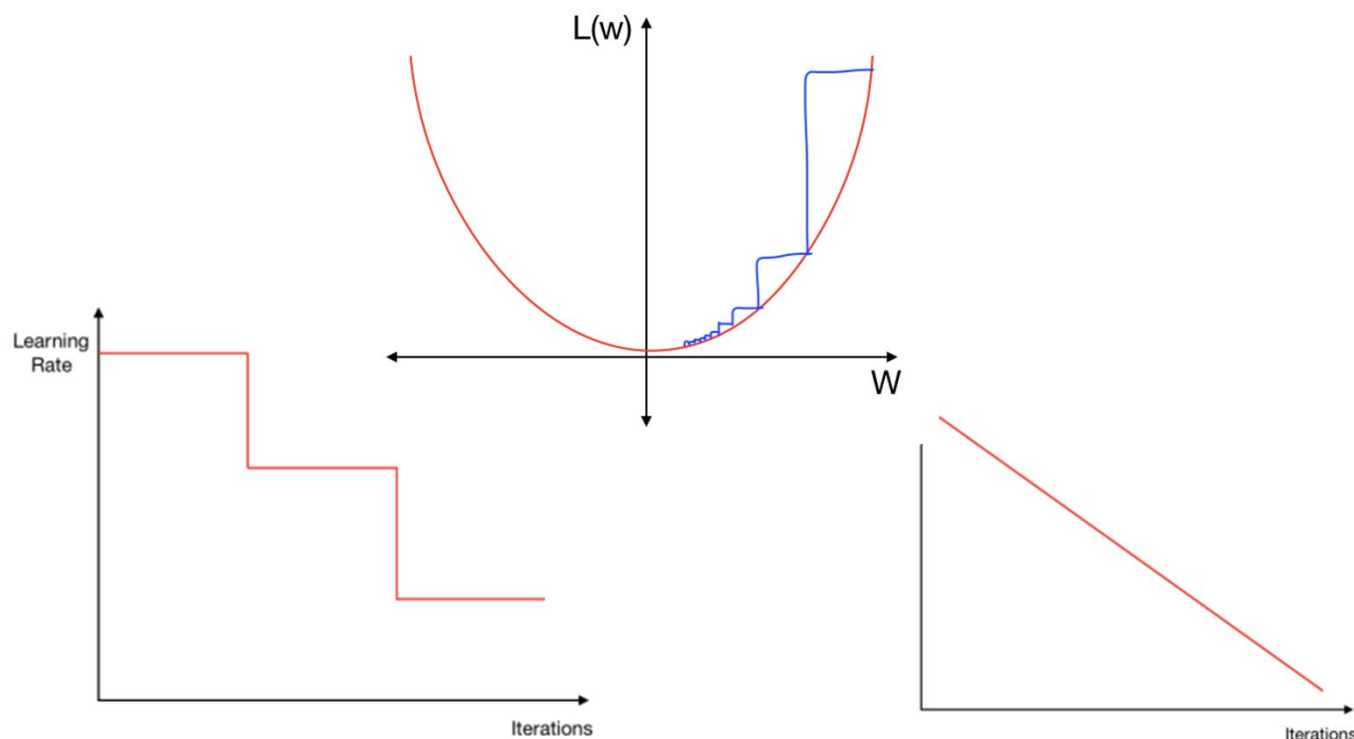
➤ 学习率是十分重要的超参数, 甚至是**最重要的超参数**



选择适当的学习率!!!

■ 学习率(Learning Rate)

➤ 学习率应适应当前的学习状态



梯级衰减(step decay)

线性衰减(Linear Decay)



■ 学习率(Learning Rate)

➤ 学习率应适应当前的学习状态

假设初始学习率为 α_0 ，在第 t 次迭代时的学习率 α_t 。常用的衰减方式可以设置为按迭代次数进行衰减。比如逆时衰减 (inverse time decay)

$$\alpha_t = \alpha_0 \frac{1}{1 + \beta \times t}, \quad (7.5)$$

或指数衰减 (exponential decay)

$$\alpha_t = \alpha_0 \beta^t, \quad (7.6)$$

或自然指数衰减 (natural exponential decay)

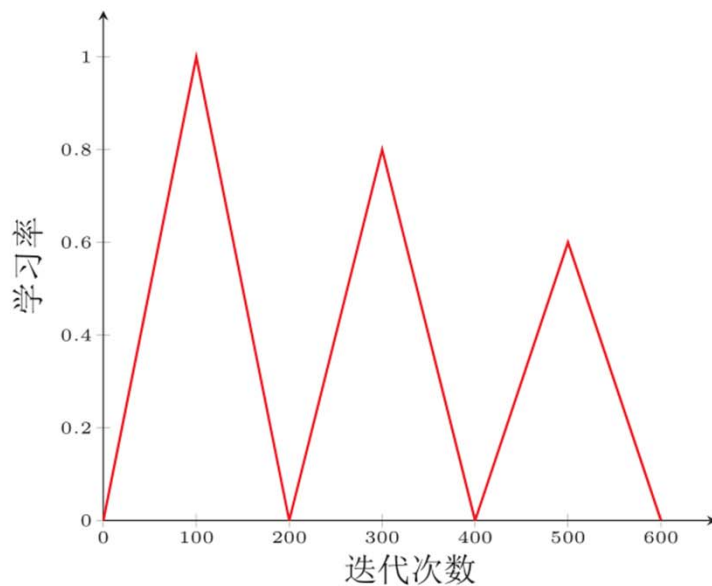
$$\alpha_t = \alpha_0 \exp(-\beta \times t), \quad (7.7)$$

其中 β 为衰减率，一般取值为0.96。

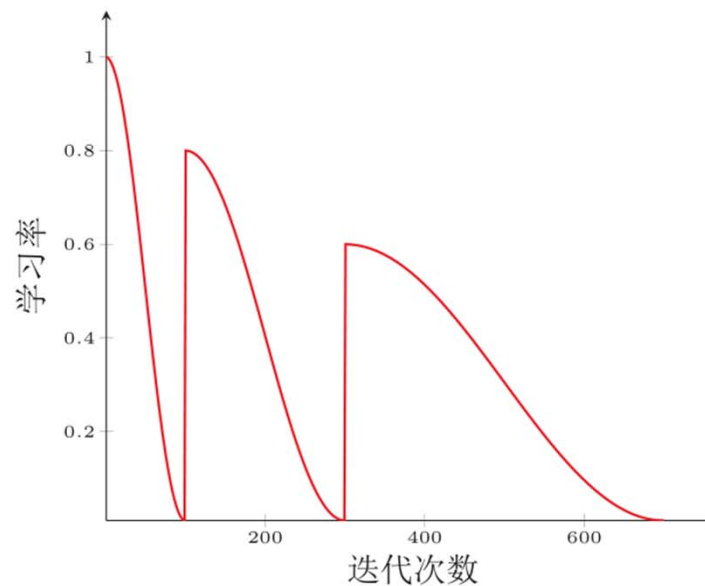


■ 学习率(Learning Rate)

➤ 周期性学习率调整



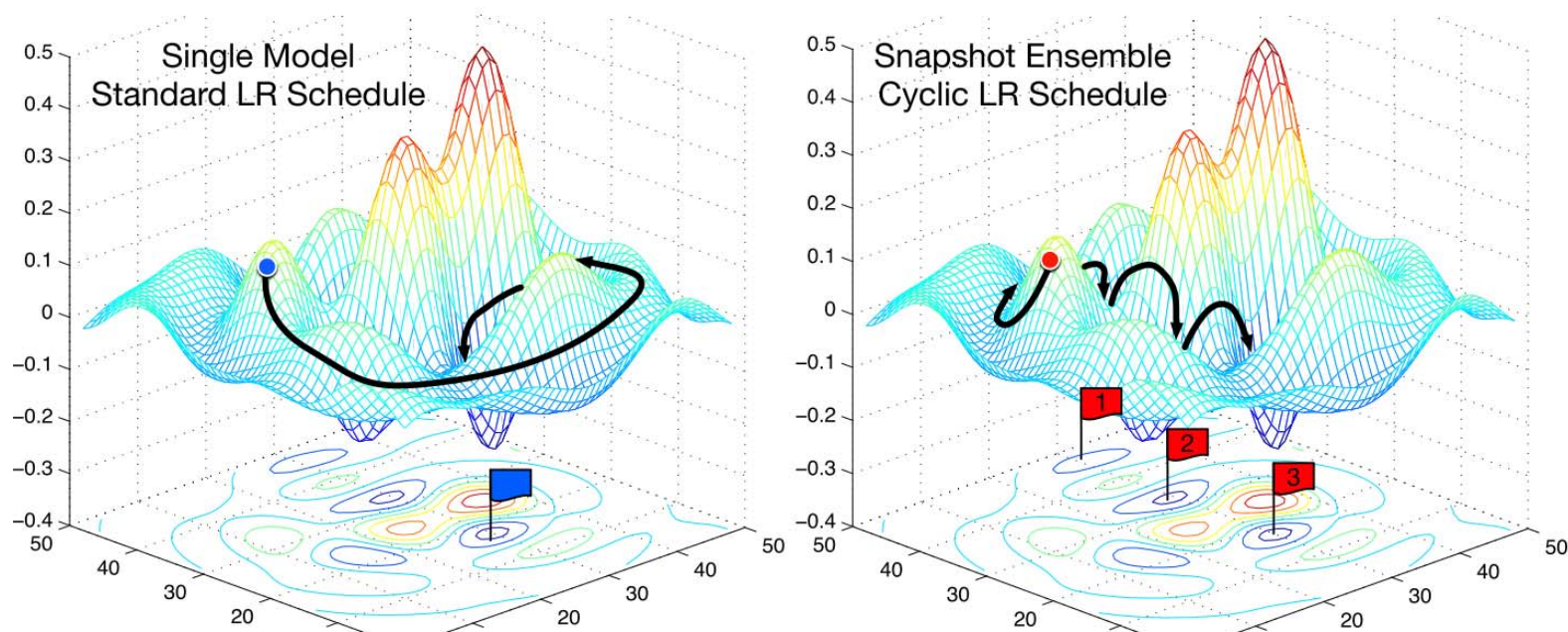
(a) 三角循环学习率



(b) 带热重启的余弦衰减

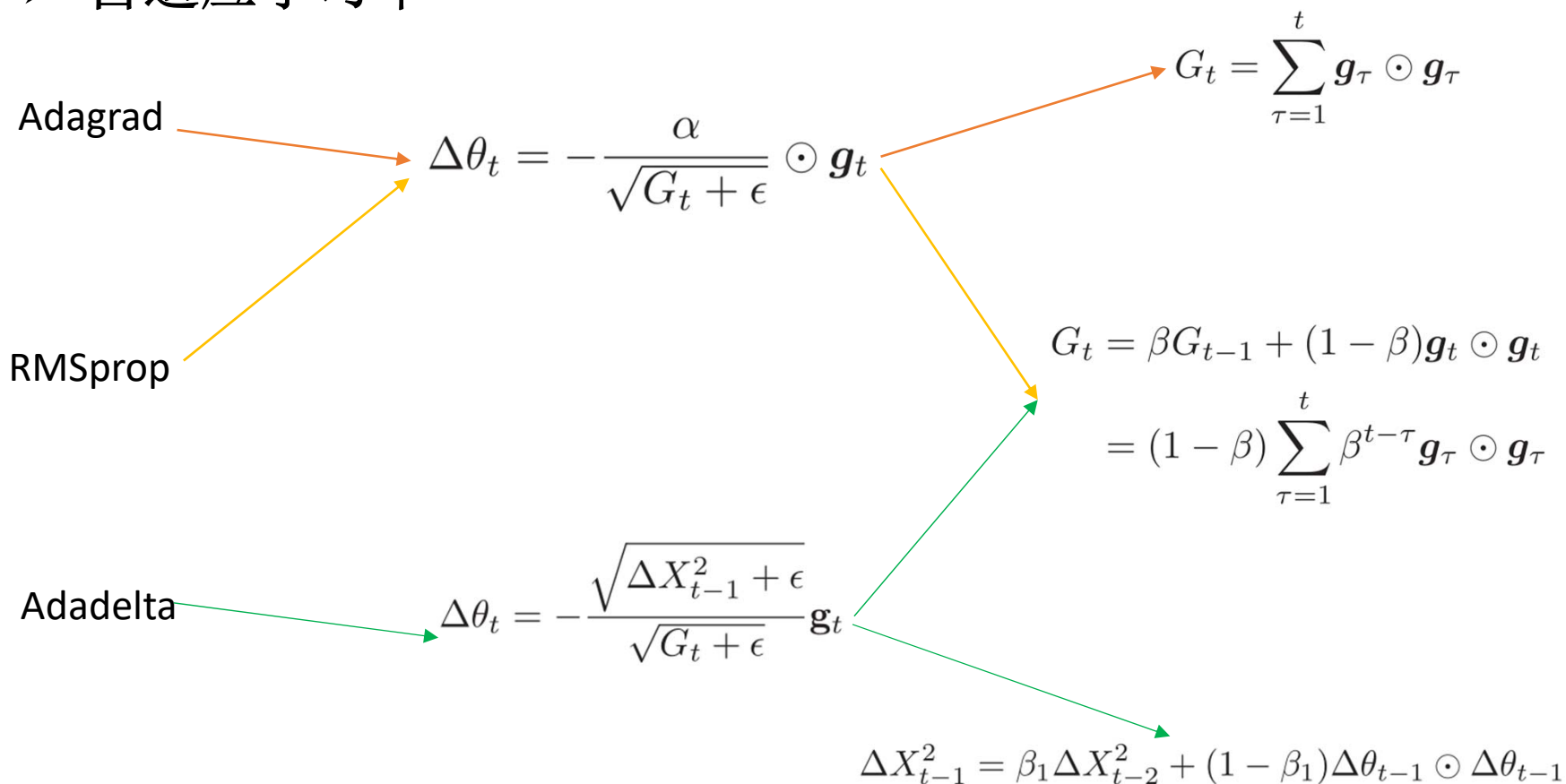
学习率(Learning Rate)

➤ 周期性学习率调整



学习率(Learning Rate)

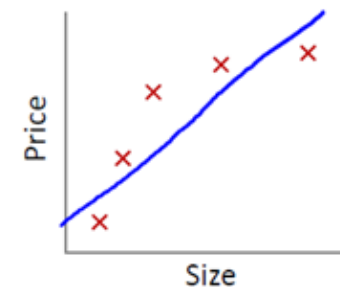
➤ 自适应学习率



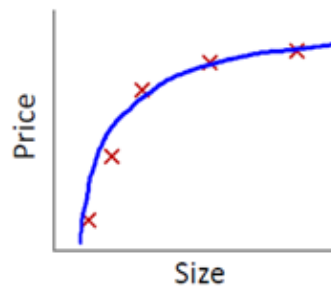
■ 正则化系数(Regularization Parameter)

- 正则化系数确保模型不**过拟合**

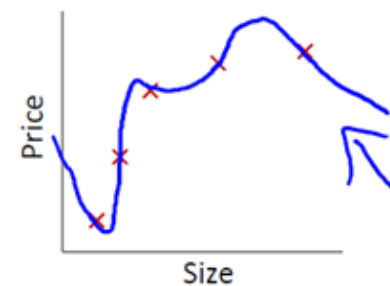
Example: Linear regression (housing prices)



$\rightarrow \theta_0 + \theta_1 x$
"Underfit" "High bias"



$\rightarrow \theta_0 + \theta_1 x + \theta_2 x^2$
"Just right"

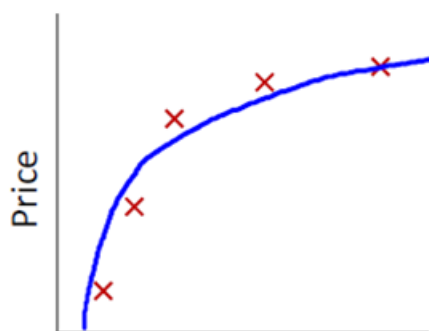


$\rightarrow \theta_0 + \theta_1 x + \theta_2 x^2 + \theta_3 x^3 + \theta_4 x^4$
"Overfit" "High variance"

Overfitting: If we have too many features, the learned hypothesis may fit the training set very well ($J(\theta) = \frac{1}{2m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2 \approx 0$), but fail to generalize to new examples (predict prices on new examples).

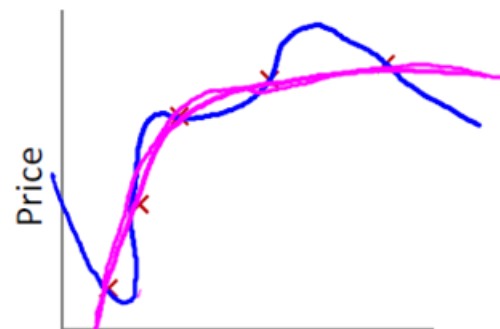
■ 正则化系数(Regularization Parameter)

Intuition



Size of house

$$\theta_0 + \theta_1 x + \theta_2 x^2$$



Size of house

$$\theta_0 + \theta_1 x + \theta_2 x^2 + \cancel{\theta_3 x^3} + \cancel{\theta_4 x^4}$$

Suppose we penalize and make θ_3, θ_4 really small.

$$\rightarrow \min_{\theta} \frac{1}{2m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2 + 1000 \theta_3^2 + 1000 \theta_4^2$$

$\theta_3 \approx 0$ $\theta_4 \approx 0$

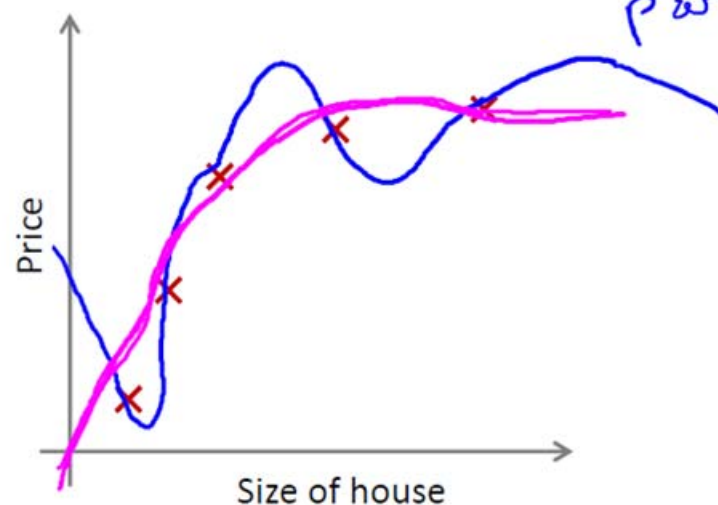
■ 正则化系数(Regularization Parameter)

Regularization.

$$\rightarrow J(\theta) = \frac{1}{2m} \left[\sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2 + \lambda \sum_{j=1}^n \theta_j^2 \right]$$

$\min_{\theta} J(\theta)$

regularization parameter



■ 正则化系数(Regularization Parameter)

➤ 建议

1. 初始正则项系数 λ 设置为0，先确定好的学习率 η
2. 固定学习率 η ，选定一个 λ 值(比如0.05)，然后根据验证集精度，将 λ 增大或者减小10倍，确定数量级
3. 当确定了 λ 的合适的数量级后，比如 $\lambda = 0.01$ ，再进一步地细调节，比如调节为0.02，0.03，0.009之类

■ 参数初始化(Parameter Initialization)

➤ 参数不能初始化为0

预训练初始化

随机初始化

固定值初始化

偏置通常用0来初始化

■ 参数初始化(Parameter Initialization)

➤ 参数不能初始化为0

对称权重问题：若所有层的权重都为0，则任何输入数据得到的输出结果都是一样的，那么反向传播算法计算出来的梯度值一样，并且参数更新值也一样($w=w-\alpha *dw$)。更一般地说，如果权重初始化为同一个值，网络就不可能不对称(即是对称的)。



你只能收敛到山谷中的一个极大值，而走不到山峰上去



■ 参数初始化(Parameter Initialization)

➤ 预训练初始化

利用已经在其他数据集上预训练的模型的权重初始化
目标模型的权重

优势:

加快模型的收敛速度

更加有可能获得低误差模型或泛化误差模型

降低初始化不当造成的梯度消失或爆炸问题



■ 参数初始化(Parameter Initialization)

➤ 随机初始化

神经元在一开始都是**随机的、独一无二的**，所以它们会计算出不同的更新，并将自己整合到整个网络的各个部分

1. **Gaussian初始化**: 参数从一个固定均值(比如0)和固定方差(比如0.01)的Gaussian分布进行随机初始化

2. **均匀分布初始化**: 参数可以在区间 $[-r, r]$ 内采用均匀分布进行初始化。

无法避免梯度消失或梯度爆炸!!!

■ 参数初始化(Parameter Initialization)

➤ 随机初始化--范数保持性

为了避免梯度消失或梯度爆炸问题，我们希望误差项

$$\|\delta^{(l-1)}\|^2 = \|\delta^{(l)}\|^2 = \|(\mathbf{W}^{(l)})^\top \delta^{(l)}\|^2$$

基于方差缩放的参数初始化

正交初始化

初始化方法	激活函数	均匀分布 $[-r, r]$	高斯分布 $\mathcal{N}(0, \sigma^2)$
Xavier 初始化	Logistic	$r = 4\sqrt{\frac{6}{M_{l-1}+M_l}}$	$\sigma^2 = 16 \times \frac{2}{M_{l-1}+M_l}$
Xavier 初始化	Tanh	$r = \sqrt{\frac{6}{M_{l-1}+M_l}}$	$\sigma^2 = \frac{2}{M_{l-1}+M_l}$
He 初始化	ReLU	$r = \sqrt{\frac{6}{M_{l-1}}}$	$\sigma^2 = \frac{2}{M_{l-1}}$

- 1) 用均值为 0、方差为 1 的高斯分布初始化一个矩阵；
- 2) 将这个矩阵用奇异值分解得到两个正交矩阵，并使用其中之一作为权重矩阵。



归一化(Normalization)

➤ 数据归一化

最小最大值归一化

标准化

PCA白化

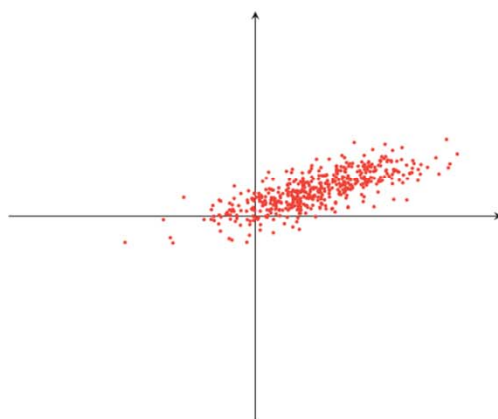
$$\frac{\max - x}{\max - \min} \text{ or } \frac{x - \min}{\max - \min}$$

$$\mu = \frac{1}{N} \sum_{n=1}^N x^{(n)},$$

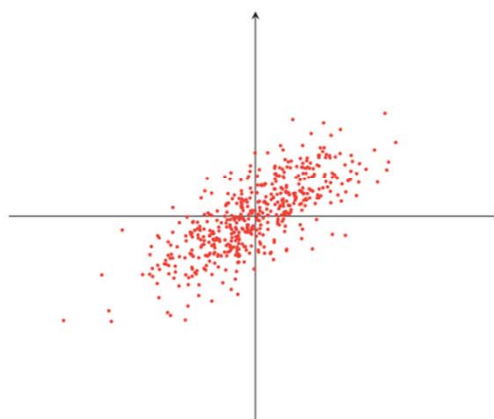
$$\sigma^2 = \frac{1}{N} \sum_{n=1}^N (x^{(n)} - \mu)^2.$$

$$\hat{x} = U \begin{bmatrix} \tilde{x}_1 \\ \vdots \\ \tilde{x}_k \\ 0 \\ \vdots \\ 0 \end{bmatrix} = \sum_{i=1}^k u_i \tilde{x}_i, \quad \frac{\sum_{j=1}^k \lambda_j}{\sum_{j=1}^n \lambda_j} \geq 0.99.$$

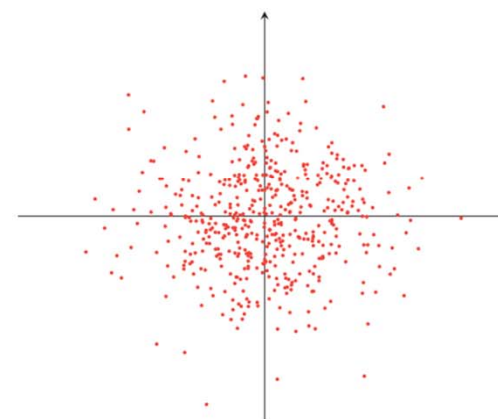
原始数据



标准归一化



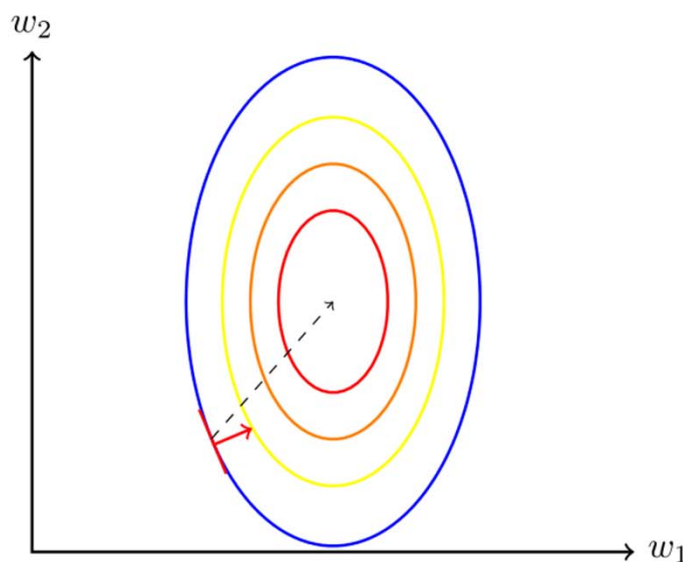
PCA 白化



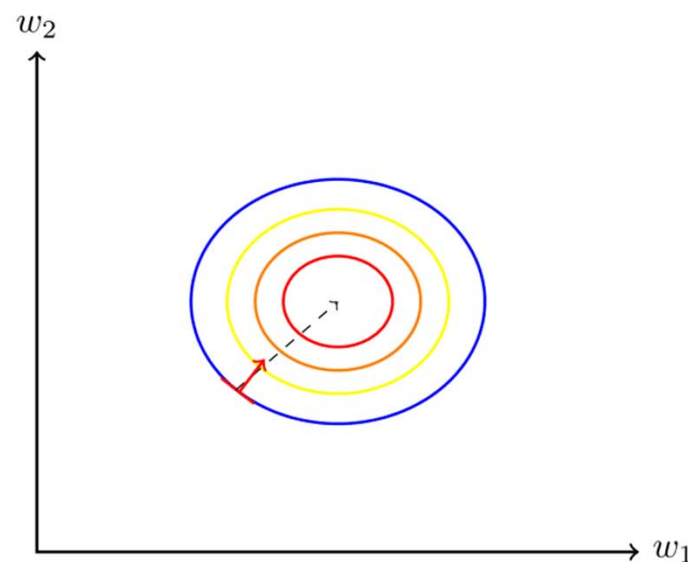
■ 归一化(Normalization)

➤ 数据归一化

数据归一化对梯度的影响



(a) 未归一化数据的梯度



(b) 归一化数据的梯度

■ 归一化(Normalization)

➤ 网络层归一化

目的： 更好的尺度不变性， 处理内部协变量偏移， 更平滑的优化曲面

归一化方法

批量归一化 (Batch Normalization)

层归一化(Layer Normalization)

示例归一化(Instance Normalization)

组归一化(Group Normalization)

权重归一化(Weight Normalization)

局部响应归一化(Local Response Normalization)



■ 归一化(Normalization)

➤ 批量归一化

对于一个深层神经网络, 令第 l 层的净输入为 $\mathbf{z}^{(l)}$, 神经元的输出为 $\mathbf{a}^{(l)}$, 即

$$\mathbf{a}^{(l)} = f(\mathbf{z}^{(l)}) = f(W\mathbf{a}^{(l-1)} + \mathbf{b}), \quad (7.42)$$

其中 $f(\cdot)$ 是激活函数, W 和 $\mathbf{b}$ 是可学习的参数。

给定一个包含 K 个样本的小批量样本集合, 计算均值和方差

$$\begin{aligned} \mu_{\mathcal{B}} &= \frac{1}{K} \sum_{k=1}^K \mathbf{z}^{(k,l)}, \\ \sigma_{\mathcal{B}}^2 &= \frac{1}{K} \sum_{k=1}^K (\mathbf{z}^{(k,l)} - \mu_{\mathcal{B}}) \odot (\mathbf{z}^{(k,l)} - \mu_{\mathcal{B}}). \end{aligned}$$

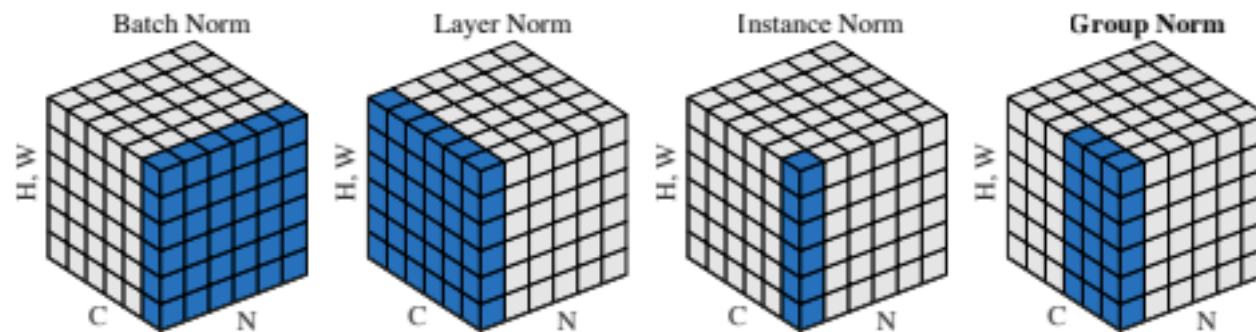
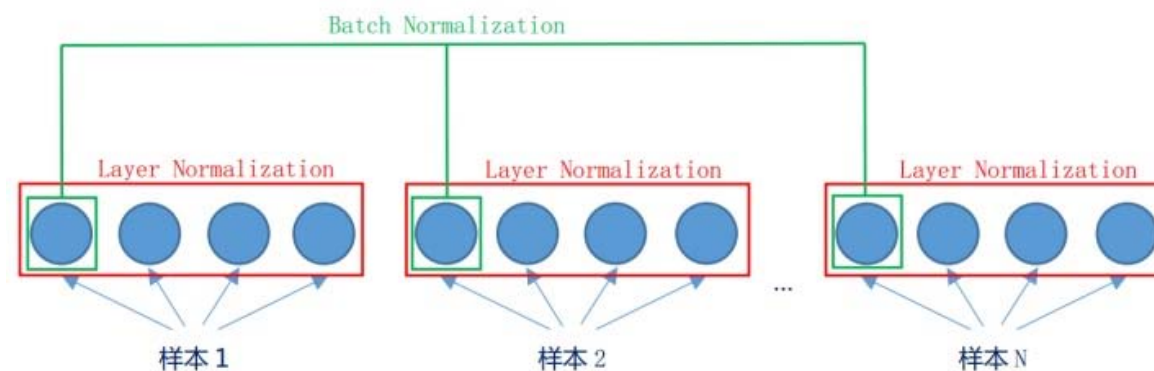
批量归一化为

$$\begin{aligned} \hat{\mathbf{z}}^{(l)} &= \frac{\mathbf{z}^{(l)} - \mu_{\mathcal{B}}}{\sqrt{\sigma_{\mathcal{B}}^2 + \epsilon}} \odot \gamma + \beta \\ &\triangleq \text{BN}_{\gamma, \beta}(\mathbf{z}^{(l)}), \end{aligned}$$



归一化(Normalization)

➤ 对比





05

PART FIVE

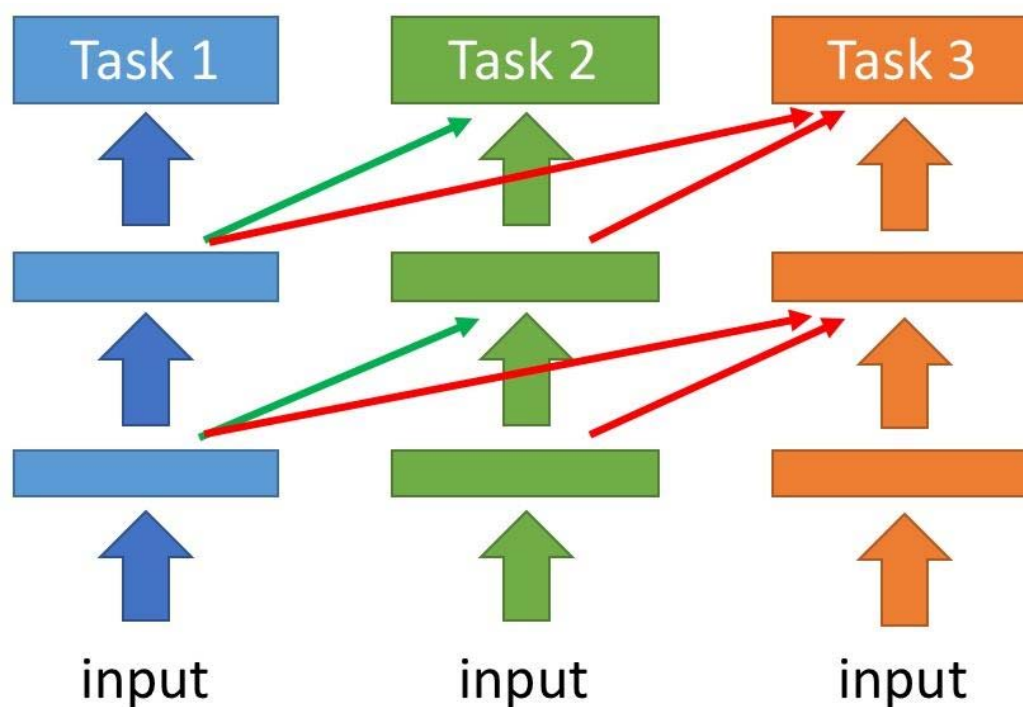
模型微调

Model Fine-tuning



■ 模型微调(Model Fine-tuning)

- 为什么要微调
- 什么情况下使用微调
- 微调的注意事项



■ 预训练模型(Pre-trained Model)

➤ 预训练模型

预先在相关数据集上训练得到的一个模型

常用的预训练模型，如VGG16/VGG19,ResNet等在大型数据集上训练好的模型(参数)

正常情况下，无需更改其他网络结构

特点：

相对于从头训练(Training from Scratch)，节省大量训练资源的训练时间，提高模型的准确率

■模型微调(Model Fine-tuning)

➤ 不做微调

从头开始训练，需要大量的数据、计算时间和资源
存在模型不收敛、参数不够优化、准确率低等问题
存在模型泛化能力差、容易过拟合等风险

➤ 使用微调

节省计算时间和资源

模型易收敛、好的参数初始化

模型泛化能力强、不容易过拟合

为什么要微调



■模型微调(Model Fine-tuning)

➤ 微调的核心动机(Motivations)

浅层提取**基础特征**，比如边缘、轮廓等

深层提取**抽象特征**，比如整个目标物体

全连接层根据**特征组合**进行语义分类

➤ 目标数据集和预训练数据集相似

➤ 从头搭建或使用的模型正确率低

➤ 数据集相似，但数据量太少

➤ 计算资源太少

什么情况下使用微调

■微调策略(Fine-tuning Strategies)

➤ 数据量少，但数据相似度高

少量的数据不足以训练模型，只能微调网络的最后几层或最终的分类层

➤ 数据量少，数据相似度低

数据相似度低，因而根据新数据集对高层进行微调更有意义;可以冻结预训练模型的初始层(如前 k 层)，然后微调剩余的层

➤ 数据量大，但数据相似度低

由于已经有大数据集，可拥有训练模型；数据相似度低，预训练模型的微调不一定有效；可以从头训练模型

➤ 数据量大，数据相似度高

预训练模型最有效，保留原始模型的结构和预训练权重，微调整个模型





PART SIX

小结

Summary





■ 小结

➤ 网络结构

- ✓ 全连接网络
- ✓ 卷积神经网络
- ✓ 循环神经网络

➤ 损失函数

- ✓ 常用损失函数

➤ 模型优化

- ✓ 学习准则, 优化问题, 梯度下降
- ✓ 目标函数, 优化求解, 多类逻辑斯蒂回归

➤ 超参技巧

- ✓ 学习率, 权重衰减, 参数初始化, 批归一化

➤ 微调方法

- ✓ 预训练模型+迁移学习
- ✓ 微调策略

